UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO COMPUTADORA

Ejercicio Práctica 3: Modelado Geométrico

Nombre de la alumna: Macias Niño Carmen Violeta

Nº de Cuenta: 318285450

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 02

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 03/03/2026

CALIFICACIÓN: _____

INDICACIONES

Instanciar primitivas geométricas para recrear el dibujo de la práctica pasada en 3D,se requiere que exista un piso; la casa tiene una ventana azul circular justo en medio de la pared trasera, 2 ventanas verdes en cada pared lateral iguales a las de la pared frontal y solo puerta en la pared frontal.

DESARROLLO

```
//practica 5: Modelado Geometrico y Camara Sintetica

#include <stdio.h>
#include <string.h>
#include <cmath>
#include <vector>

#include <glew.h>
#include <glfw3.h>

//glm
#include <glm.hpp>
#include <gtc\matrix_transform.hpp>
#include <gtc\type_ptr.hpp>
#include <gtc\random.hpp>

//clases para dar orden y limpieza al còdigo
#include "Mesh.h"
#include "Shader.h"
#include "Sphere.h"
#include "Window.h"
#include "Camera.h"
```

Includes (librerías y dependencias)

- **cmath / PI / cos / sin**: lo uso para generar figuras como cilindro y cono con trigonometría.
- **vector**: lo uso para construir arreglos dinámicos de vértices e índices.
- **GLEW**: me permite cargar extensiones de OpenGL modernas (VAO, VBO, shaders, etc.).
- **GLFW**: crea la ventana y administra eventos de teclado/mouse.
- **GLM**: es mi “calculadora” de álgebra lineal: matrices 4x4, transformaciones (translate/rotate/scale) y cámara (lookAt).
- Las clases **Mesh / Shader / Sphere / Window / Camera** son mi estructura para no tener todo en un solo archivo: cada una encapsula una parte del sistema (geometría, shaders, ventana y cámara).

Variables globales de control

```
// Constantes y variables globales
const float toRadians = 3.14159265f / 180.0f;      // grados a radianes
const float PI = 3.14159265f;                      // PI para trigonometria
GLfloat deltaTime = 0.0f;                          // deltaTime para movimiento suave
GLfloat lastTime = 0.0f;                          // tiempo del frame anterior
static double limitFPS = 1.0 / 60.0;               // intento de estabilizar la actualización

Camera camera;                                     // cámara sintética
Window mainWindow;                                 // ventana
vector<Mesh*> meshList;                            // lista de mallas
vector<Shader> shaderList;                         // lista de shaders
```

Este bloque define lo que necesito para que el programa corra “en tiempo real”:

- **deltaTime**: mide cuánto tiempo pasó entre un frame y otro. Esto es clave porque así la cámara se mueve igual de rápido aunque mi PC vaya a 30fps o 120fps.
- **meshList**: almacena las primitivas creadas (cubo, cilindro, cono, techo, etc.).
- **shaderList**: almacena los shaders compilados.
- **camera** y **mainWindow** son objetos globales para poder usarlos fácil dentro del render loop.

Rutas de shaders y creación de la esfera

```
// Shaders (rutas)
static const char* vShader = "shaders/shader.vert";
static const char* fShader = "shaders/shader.frag";
static const char* vShaderColor = "shaders/shadercolor.vert";

// Esfera (para usarla como "ventana circular" aplastada)
// NOTA: si tu Sphere.cpp pinta con colores aleatorios por vértice,
//       la ventana podría no verse azul. Abajo te doy una solución rápida.
Sphere sp = Sphere(1.0f, 20, 20);                    // radio, slices, stacks
```

Aquí defino las rutas a mis shaders (vertex y fragment shader).
También creo `sp`, una esfera con radio 1 y resolución 20x20. En la escena la uso **aplastada** para simular una **ventana circular** en la parte trasera.

PRIMITIVAS (MALLAS)

```
void CrearCubo()
{
    unsigned int cubo_indices[] = {
        // front
        0, 1, 2,
        2, 3, 0,
        // right
        1, 5, 6,
        6, 2, 1,
        // back
        7, 6, 5,
        5, 4, 7,
        // left
        4, 0, 3,
        3, 7, 4,
        // bottom
        4, 5, 1,
        1, 0, 4,
        // top
        3, 2, 6,
        6, 7, 3
    };

    GLfloat cubo_vertices[] = {
        // front
        -0.5f, -0.5f,  0.5f,
         0.5f, -0.5f,  0.5f,
         0.5f,  0.5f,  0.5f,
        -0.5f,  0.5f,  0.5f,
        // back
        -0.5f, -0.5f, -0.5f,
         0.5f, -0.5f, -0.5f,
         0.5f,  0.5f, -0.5f,
        -0.5f,  0.5f, -0.5f
    };

    Mesh* cubo = new Mesh();
    cubo->CreateMesh(cubo_vertices, cubo_indices, 24, 36);
    meshList.push_back(cubo);
}
```

En `CrearCubo()` defino:

- `cubo_vertices`: 8 vértices (coordenadas xyz).
- `cubo_indices`: índices que conectan esos puntos formando 12 triángulos (2 por cara).

CrearPiramideTriangular()

Esta figura realmente no se usa en la casa final, pero se deja como parte del repertorio de primitivas de la práctica. Es una pirámide de base triangular, definida por vértices e índices.

CrearCilindro(res, R)

Este bloque es importante porque el cilindro se usa como **tronco** del árbol.

- res controla cuántas "rebanadas" tiene el cilindro (más rebanadas → más redondo).
- R es el radio.

```
// Pirámide triangular regular (no se usa en la casa final, pero se conserva)
void CrearPiramideTriangular()
{
    unsigned int indices_piramide_triangular[] = {
        0,1,2,
        1,3,2,
        3,0,2,
        1,0,3
    };

    GLfloat vertices_piramide_triangular[] = {
        -0.5f, -0.5f,  0.0f,   //0
         0.5f, -0.5f,  0.0f,   //1
         0.0f,  0.5f, -0.25f,  //2
         0.0f, -0.5f, -0.5f    //3
    };

    Mesh* obj1 = new Mesh();
    obj1->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
    meshList.push_back(obj1);
```

```
    }
    // circunferencia inferior
    for (n = 0; n <= res; n++)
    {
        x = R * cos(n * dt);
        z = R * sin(n * dt);

        vertices.push_back(x);
        vertices.push_back(-0.5f);
        vertices.push_back(z);
    }
    // circunferencia superior
    for (n = 0; n <= res; n++)
    {
        x = R * cos(n * dt);
        z = R * sin(n * dt);

        vertices.push_back(x);
        vertices.push_back(0.5f);
        vertices.push_back(z);
    }
    // ◦ Generar índices correctamente (uno por vértice)
    for (unsigned int k = 0; k < vertices.size() / 3; k++)
    {
        indices.push_back(k);
    }
    // Crear malla
    Mesh* cilindro = new Mesh();
    cilindro->CreateMeshGeometry(vertices, indices,
        (unsigned int)vertices.size(),
        (unsigned int)indices.size());
    meshList.push_back(cilindro);
}
```

Primero genero vértices usando trigonometría:

- Para cada ángulo theta, calculo:
  - x = R*cos(theta)
  - z = R*sin(theta)
- Para cada rebanada creo dos puntos:
  - un punto abajo (y = -0.5)
  - y uno arriba (y = 0.5)

Después genero vértices extra para circunferencia inferior y superior (aunque en este caso no estoy triangulando tapas, es más bien un remanente de la versión original).

```
void CrearCilindro(int res, float R)
{
    int n;
    GLfloat dt = 2 * PI / res, x, z, y = -0.5f;

    vector<GLfloat> vertices;
    vector<unsigned int> indices;

    // paredes del cilindro: por cada "rebanada" agregas 2 vértices (abajo y arriba)
    for (n = 0; n <= res; n++)
    {
        if (n != res) {
            x = R * cos(n * dt);
            z = R * sin(n * dt);
        }
        else {
            x = R * cos(0 * dt);   // cerrar el círculo
            z = R * sin(0 * dt);
        }
        // vértice inferior
        vertices.push_back(x);
        vertices.push_back(y);
        vertices.push_back(z);
        // vértice superior
        vertices.push_back(x);
        vertices.push_back(0.5f);
        vertices.push_back(z);
    }
    // circunferencia inferior
    for (n = 0; n <= res; n++)
    {
        x = R * cos(n * dt);
        z = R * sin(n * dt);
```

CrearCono(res, R)

Este se usa como **copa del árbol**.

- El primer vértice es la punta del cono.
- Luego genero la circunferencia de la base con cos/sin.
- Los índices se generan secuenciales para que el cono se dibuje como una especie de abanico de triángulos desde la punta.

```
void CrearCono(int res, float R)
{
    int n, i;
    GLfloat dt = 2 * PI / res, x, z, y = -0.5f;

    vector<GLfloat> vertices;
    vector<unsigned int> indices;

    // punta del cono
    vertices.push_back(0.0f);
    vertices.push_back(0.5f);
    vertices.push_back(0.0f);

    // circunferencia de base
    for (n = 0; n <= res; n++)
    {
        x = R * cos((n)*dt);
        z = R * sin((n)*dt);

        vertices.push_back(x);
        vertices.push_back(y);
        vertices.push_back(z);
    }

    // cierre del circulo
    vertices.push_back(R * cos(0) * dt);
    vertices.push_back(-0.5f);
    vertices.push_back(R * sin(0) * dt);

    for (i = 0; i < res + 2; i++) indices.push_back(i);

    Mesh* cono = new Mesh();
    cono->CreateMeshGeometry(vertices, indices, (unsigned int)vertices.size(), (unsigned int)(res + 2));
    meshList.push_back(cono);
}
```

CrearPiramideCuadrangular()

Esta malla se usa como **techo**.
Tiene 5 vértices: 4 de la base cuadrada y 1 punta.
Los índices forman triángulos que conectan la punta con cada lado, y también la base.

```
void CrearPiramideCuadrangular()
{
    vector<unsigned int> piramidecuadrangular_indices = {
        0,3,4,
        3,2,4,
        2,1,4,
        1,0,4,
        0,1,2,
        0,2,4
    };

    vector<GLfloat> piramidecuadrangular_vertices = {
         0.5f,-0.5f, 0.5f,
         0.5f,-0.5f,-0.5f,
        -0.5f,-0.5f,-0.5f,
        -0.5f,-0.5f, 0.5f,
         0.0f, 0.5f, 0.0f,
    };

    Mesh* piramide = new Mesh();
    piramide->CreateMeshGeometry(piramidecuadrangular_vertices, piramidecuadrangular_indices, 15, 18);
    meshList.push_back(piramide);
}
```

CreateShaders()

En este bloque instancio dos shaders y los compilo desde archivos:

```
void CreateShaders()
{
    Shader* shader1 = new Shader();
    shader1->CreateFromFiles(vShader, fShader);
    shaderList.push_back(*shader1);

    Shader* shader2 = new Shader();
    shader2->CreateFromFiles(vShaderColor, fShader);
    shaderList.push_back(*shader2);
}
```

Esto me permite usar `useShader()` y luego consultar las ubicaciones de uniforms como:

- model
- projection
- view
- color

MAIN

Inicialización de ventana + Depth Test

Activo **Depth Test** para que OpenGL renderice correctamente qué está adelante y qué está atrás, usando el buffer de profundidad.

Sin esto, el orden de dibujo sería el que manda y la escena se vería incorrecta.

```
int main()
{
    mainWindow = Window(800, 600);
    mainWindow.Initialise();

    glEnable(GL_DEPTH_TEST);   // Activa prueba de profundidad (Z-buffer)
    glDepthFunc(GL_LESS);      // (Opcional pero recomendado) acepta fragmentos más cercanos

    // Crear primitivas
    CrearCubo();                // meshList[0] -> cubo (RenderMesh)
    CrearPiramideTriangular();  // meshList[1] -> pirámide triangular
    CrearCilindro(60, 1.0f);    // meshList[2] -> cilindro (RenderMeshGeometry) (res=20 para troncos suaves)
    CrearCono(30, 1.0f);        // meshList[3] -> cono (RenderMeshGeometry) (res=30 para copas suaves)
    CrearPiramideCuadrangular(); // meshList[4] -> techo (RenderMeshGeometry)

    CreateShaders();
```

Creación de primitivas e inicialización de cámara
Aquí creo mis mallas con las resoluciones que quiero:

- Cilindro con **60** para que el tronco se vea redondo.
- Cono con **30** para que la copa también se vea suave.

Después creo la cámara:

```
camera = Camera(position, up, yaw, pitch, moveSpeed, turnSpeed);
```

Eso significa que puedo moverme con WASD y girar con mouse.

Inicialización y carga de la esfera

```
sp.init();
sp.load();
```

- `init()` genera los vértices/índices.
- `load()` crea VAO/VBO/EBO y los manda a GPU.

```
296
297         // PROYECCIÓN (FIX: aspect ratio en float)
298         glm::mat4 projection = glm::perspective(glm::radians(60.0f),
299             (float)mainWindow.getBufferWidth() / (float)mainWindow.getBufferHeight(), // <-- aspect correcto
300             0.1f, 100.0f);
301
302         // Inicializar y cargar esfera (para ventana circular)
303         sp.init();      // genera malla (vértices/índices)
304         sp.load();      // manda buffers a la GPU
305
306         glm::mat4 model(1.0f);                  // matriz modelo
307         glm::vec3 color(0.0f, 0.0f, 0.0f);      // color para uniform
```

LOOP PRINCIPAL DE RENDER

Control de tiempo + eventos

Cada frame calculo `deltaTime`, leo eventos y actualizo cámara:

```
glfwPollEvents();
camera.keyControl(...);
camera.mouseControl(...);
```

Eso hace que el movimiento sea continuo y responsivo.

```
// Loop principal
while (!mainWindow.getShouldClose())
{
    // Tiempo
    GLfloat now = (GLfloat)glfwGetTime();
    deltaTime = now - lastTime;
    deltaTime += (now - lastTime) / (GLfloat)limitFPS;
    lastTime = now;

    // Eventos
    glfwPollEvents();

    // Cámara (WASD + mouse)
    camera.keyControl(mainWindow.getsKeys(), deltaTime);
    camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());

    // Limpiar
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);                    // fondo blanco como tu ejemplo 2D
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Usar shader principal (uniformColor)
    shaderList[0].useShader();
    uniformModel = shaderList[0].getModelLocation();
    uniformProjection = shaderList[0].getProjectLocation();
    uniformView = shaderList[0].getViewLocation();
    uniformColor = shaderList[0].getColorLocation();

    // Enviar Projection y View (una vez por frame)
    glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
    glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));

    // Rotación global por teclas E/R/T (según tu Window)
    float rotX = (float)mainWindow.getrotax();                // rotación X global
    float rotY = (float)mainWindow.getrotay();                // rotación Y global
    float rotZ = (float)mainWindow.getrotaz();                // rotación Z global
```

Limpiar buffers

```
glClearColor(1,1,1,1);
glClear(GL_COLOR_BUFFER_BIT |
GL_DEPTH_BUFFER_BIT);
```

Pongo fondo blanco como en el ejemplo de la casa 2D y limpio también profundidad.

Activar shader y obtener uniforms

```
shaderList[0].useShader();
uniformModel = ...
uniformProjection = ...
uniformView = ...
uniformColor = ...
```

Estoy indicando: “voy a renderizar usando este shader”, y guardo los IDs de los uniforms para poder mandar matrices y colores.

Enviar projection y view

```
glUniformMatrix4fv(uniformProjection,...);
glUniformMatrix4fv(uniformView,...);
```

- `projection` define la cámara (perspectiva).
- `view` viene de `camera.calculateViewMatrix()` y representa la posición/orientación del observador.

Rotación global E/R/T

```
float rotX = mainWindow.getrotax();
...
```

Estas rotaciones permiten girar toda la escena (útil para ver los 4 lados de la casa como en la imagen).

Lambdas: DrawCube, DrawGeom, DrawSphereDisk

Aquí creo funciones auxiliares locales para no repetir código:

```
//=====================================================
// FUNCIONES AUXILIARES (LAMBDA) PARA INSTANCIAR OBJETOS
//=====================================================

// Dibujar un cubo (meshList[0]) usando RenderMesh()
auto DrawCube = [&](glm::vec3 pos, glm::vec3 scl, glm::vec3 col)
    {
        model = glm::mat4(1.0f);                                          // reiniciar Model
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, -6.0f));      // alejar escena en -Z
        model = glm::rotate(model, glm::radians(rotX), glm::vec3(1, 0, 0));   // rotación global X
        model = glm::rotate(model, glm::radians(rotY), glm::vec3(0, 1, 0));   // rotación global Y
        model = glm::rotate(model, glm::radians(rotZ), glm::vec3(0, 0, 1));   // rotación global Z
        model = glm::translate(model, pos);                               // posicionar objeto
        model = glm::scale(model, scl);                                   // escalar objeto

        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));   // enviar Model
        glUniform3fv(uniformColor, 1, glm::value_ptr(col));               // enviar Color
        meshList[0]->RenderMesh();                                        // dibujar cubo
    };
```

DrawCube

Sirve para dibujar todo lo que sea cubo (piso, casa, puerta, ventanas).

Hace:

1. reiniciar model
2. alejar escena a -Z
3. aplicar rotaciones globales

4. translate a la posición del objeto
5. scale al tamaño
6. enviar `model` y `color`
7. render del cubo

```
// Dibujar geometría creada con CreateMeshGeometry (cilindro/cono/pirámide)
auto DrawGeom = [&](int meshIndex, glm::vec3 pos, glm::vec3 scl, glm::vec3 rotDeg, glm::vec3 col)
    {
        model = glm::mat4(1.0f);                                                    // reiniciar Model
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, -6.0f));                // alejar escena en -Z
        model = glm::rotate(model, glm::radians(rotX), glm::vec3(1, 0, 0));           // rotación global X
        model = glm::rotate(model, glm::radians(rotY), glm::vec3(0, 1, 0));           // rotación global Y
        model = glm::rotate(model, glm::radians(rotZ), glm::vec3(0, 0, 1));           // rotación global Z
        model = glm::translate(model, pos);                                         // posición
        model = glm::rotate(model, glm::radians(rotDeg.x), glm::vec3(1, 0, 0));       // rotación local X
        model = glm::rotate(model, glm::radians(rotDeg.y), glm::vec3(0, 1, 0));       // rotación local Y
        model = glm::rotate(model, glm::radians(rotDeg.z), glm::vec3(0, 0, 1));       // rotación local Z
        model = glm::scale(model, scl);                                             // escala

        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));       // enviar Model
        glUniform3fv(uniformColor, 1, glm::value_ptr(col));                         // enviar Color
        meshList[meshIndex]->RenderMeshGeometry();                                  // dibujar geometría
    };
```

DrawGeom

Hace lo mismo pero para cilindro, cono y techo (mallas con CreateMeshGeometry).

DrawSphereDisk

Sirve para la ventana circular: pongo la esfera en la pared trasera y la **aplasto** en Z para que se vea como disco.

```
// Dibujar esfera (para ventana circular trasera) aplastada a disco
auto DrawSphereDisk = [&](glm::vec3 pos, glm::vec3 scl, glm::vec3 col)
    {
        model = glm::mat4(1.0f);                                                    // reiniciar Model
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, -6.0f));                // alejar escena en -Z
        model = glm::rotate(model, glm::radians(rotX), glm::vec3(1, 0, 0));           // rotación global X
        model = glm::rotate(model, glm::radians(rotY), glm::vec3(0, 1, 0));           // rotación global Y
        model = glm::rotate(model, glm::radians(rotZ), glm::vec3(0, 0, 1));           // rotación global Z
        model = glm::translate(model, pos);                                         // posicionar
        model = glm::scale(model, scl);                                             // aplastar (disco)

        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));       // enviar Model
        glUniform3fv(uniformColor, 1, glm::value_ptr(col));                         // intentar forzar color azul (depende shader)
        sp.render();                                                                // dibujar esfera
    };
```

Definición de colores

Defino colores tipo “paleta” para que todo sea consistente:

- casa roja
- techo azul
- ventanas verdes
- tronco café
- copa verde oscuro
- piso gris

```
//=====================================================
// COLORES (como tu dibujo)
//=====================================================
glm::vec3 colPiso = glm::vec3(0.85f, 0.85f, 0.85f);   // gris claro
glm::vec3 colCasa = glm::vec3(1.00f, 0.00f, 0.00f);   // rojo
glm::vec3 colTecho = glm::vec3(0.00f, 0.00f, 1.00f);   // azul
glm::vec3 colVentana = glm::vec3(0.00f, 1.00f, 0.00f);   // verde
glm::vec3 colPuerta = glm::vec3(0.00f, 1.00f, 0.00f);   // verde
glm::vec3 colMarco = glm::vec3(0.10f, 0.10f, 0.10f);   // casi negro (marco)
glm::vec3 colTronco = glm::vec3(0.40f, 0.30f, 0.10f);   // café
glm::vec3 colCopa = glm::vec3(0.00f, 0.50f, 0.00f);   // verde oscuro
glm::vec3 colCirculo = glm::vec3(0.00f, 0.00f, 1.00f);   // azul (ventana circular)
```

Cálculo de dimensiones de la casa

Esto es clave: calculo posiciones de caras:

- `zFront`, `zBack`: para saber dónde está la pared frontal y trasera.
- `xRight`, `xLeft`: paredes laterales.

```
//=====================================================
// DIMENSIONES BASE DE LA CASA (para ubicar ventanas/puerta)
//=====================================================
glm::vec3 casaScale = glm::vec3(2.0f, 2.4f, 1.6f);        // escala del cuerpo de la casa (cubo)
glm::vec3 casaPos = glm::vec3(0.0f, 0.2f, 0.0f);        // posición del centro de la casa

float halfW = 0.5f * casaScale.x;                         // mitad del ancho
float halfH = 0.5f * casaScale.y;                         // mitad de la altura
float halfD = 0.5f * casaScale.z;                         // mitad de la profundidad

float zFront = casaPos.z + halfD;                         // cara frontal (z +)
float zBack = casaPos.z - halfD;                         // cara trasera (z -)
float xRight = casaPos.x + halfW;                         // pared derecha (x +)
float xLeft = casaPos.x - halfW;                         // pared izquierda (x -)

float epsOut = 0.03f;                                     // "sobresalir muy poco" (marcos)
float epsObj = 0.02f;                                     // ventana/puerta sobresale poquito
```

También defino `epsObj` y `epsOut` para que las ventanas/puerta y sus marcos sobresalgan poquito sin meterse dentro de la pared.

Instanciación de la escena

En este orden renderizo:

1. Piso (cubo plano)
2. Cuerpo casa (cubo escalado)
3. Techo (pirámide)
4. Puerta + marco
5. Ventanas frontales + marcos
6. Ventanas laterales + marcos
7. Ventana circular trasera (esfera aplastada)
8. Árboles (tronco cilindro + copa cono) a izquierda y derecha

```
//=====================================================
// 1) PISO (cubo muy plano)
//=====================================================
DrawCube(glm::vec3(0.0f, -0.95f, 0.0f),                    // posición del piso
    glm::vec3(8.0f, 0.15f, 8.0f),                         // escala grande y plano
    colPiso);                                             // color del piso

//=====================================================
// 2) CUERPO DE LA CASA (cubo rojo)
//=====================================================
DrawCube(casaPos, casaScale, colCasa);

//=====================================================
// 3) TECHO (pirámide cuadrangular azul)
//    Colocarlo encima de la casa
//=====================================================
DrawGeom(4,                                                // meshList[4] -> pirámide cuadrangular
    glm::vec3(casaPos.x, casaPos.y + halfH + 0.55f, casaPos.z), // arriba de la casa
    glm::vec3(casaScale.x * 1.05f, 1.4f, casaScale.z * 1.05f),  // un poco más grande que la casa
    glm::vec3(0.0f, 0.0f, 0.0f),                          // sin rotación local
    colTecho);                                            // azul
```

Eso recrea exactamente el modelo de tu dibujo.

```
//=====================================================
// 4) PUERTA (solo frontal) + MARCO sobresale poquito
//=====================================================
// Puerta (verde)
DrawCube(glm::vec3(0.0f, casaPos.y - 0.65f, zFront + epsObj), // centrada y abajo en la pared frontal
    glm::vec3(0.55f, 0.85f, 0.08f),                       // delgada en Z para pegarse a la pared
    colPuerta);

// Marco de puerta (negro) sobresale "muy poco"
DrawCube(glm::vec3(0.0f, casaPos.y - 0.65f, zFront + epsOut), // casi mismo lugar, un poquito más afuera
    glm::vec3(0.62f, 0.92f, 0.05f),                       // ligeramente más grande
    colMarco);

//=====================================================
// 5) VENTANAS FRONTales (2) + MARCOS
//=====================================================
// Ventana frontal izquierda (verde)
DrawCube(glm::vec3(-0.55f, casaPos.y + 0.45f, zFront + epsObj),
    glm::vec3(0.55f, 0.55f, 0.08f),
    colVentana);

// Marco frontal izquierda
DrawCube(glm::vec3(-0.55f, casaPos.y + 0.45f, zFront + epsOut),
    glm::vec3(0.62f, 0.62f, 0.05f),
    colMarco);

// Ventana frontal derecha (verde)
DrawCube(glm::vec3(0.55f, casaPos.y + 0.45f, zFront + epsObj),
    glm::vec3(0.55f, 0.55f, 0.08f),
    colVentana);

// Marco frontal derecha
DrawCube(glm::vec3(0.55f, casaPos.y + 0.45f, zFront + epsOut),
    glm::vec3(0.62f, 0.62f, 0.05f),
    colMarco);
```

```
//=========================================================
// 6) VENTANAS LATERALES: 2 en pared derecha + 2 en pared izquierda
//    - Mismas que las frontales (mismo tamaño), solo pegadas a paredes X
//=========================================================
// Pared derecha (x +): hacemos cubos muy delgados en X
// Ventana derecha 1
DrawCube(glm::vec3(xRight + epsObj, casaPos.y + 0.45f, -0.35f),
    glm::vec3(0.08f, 0.55f, 0.55f),
    colVentana);
// Marco derecha 1
DrawCube(glm::vec3(xRight + epsOut, casaPos.y + 0.45f, -0.35f),
    glm::vec3(0.05f, 0.62f, 0.62f),
    colMarco);

// Ventana derecha 2
DrawCube(glm::vec3(xRight + epsObj, casaPos.y + 0.45f, 0.35f),
    glm::vec3(0.08f, 0.55f, 0.55f),
    colVentana);
// Marco derecha 2
DrawCube(glm::vec3(xRight + epsOut, casaPos.y + 0.45f, 0.35f),
    glm::vec3(0.05f, 0.62f, 0.62f),
    colMarco);

// Pared izquierda (x -)
// Ventana izquierda 1
DrawCube(glm::vec3(xLeft - epsObj, casaPos.y + 0.45f, -0.35f),
    glm::vec3(0.08f, 0.55f, 0.55f),
    colVentana);
// Marco izquierda 1
DrawCube(glm::vec3(xLeft - epsOut, casaPos.y + 0.45f, -0.35f),
    glm::vec3(0.05f, 0.62f, 0.62f),
    colMarco);

// Ventana izquierda 2
DrawCube(glm::vec3(xLeft - epsObj, casaPos.y + 0.45f, 0.35f),
    glm::vec3(0.08f, 0.55f, 0.55f),
    colVentana);
// Marco izquierda 2
DrawCube(glm::vec3(xLeft - epsOut, casaPos.y + 0.45f, 0.35f),
    glm::vec3(0.05f, 0.62f, 0.62f),
    colMarco);
```

```
        //=========================================================
        // 7) VENTANA CIRCULAR AZUL TRASERA (centrada)
        //    - Se usa esfera aplastada en Z para simular un disco circular.
        //=========================================================
        DrawSphereDisk(glm::vec3(0.0f, casaPos.y + 0.05f, zBack - epsObj),  // centrada en pared trasera
            glm::vec3(0.75f, 0.95f, 0.10f),                                 // aplastar para "disco"
            colCirculo);                                                    // azul

        //=========================================================
        // 8) ÁRBOLES (como tu ejemplo)
        //    - Tronco: cilindro (meshList[2])
        //    - Copa: cono (meshList[3])
        //=========================================================
        auto DrawTree = [&](glm::vec3 basePos)
            {
                // Tronco (cilindro)
                DrawGeom(2,                                                         // meshList[2] -> cilindro
                    glm::vec3(basePos.x, basePos.y - 0.55f, basePos.z),     // baja un poco para que toque pis
                    glm::vec3(0.25f, 0.9f, 0.25f),                          // tronco delgado y alto
                    glm::vec3(0.0f, 0.0f, 0.0f),                            // sin rotación local
                    colTronco);                                             // color tronco

                // Copa (cono)
                DrawGeom(3,                                                         // meshList[3] -> cono
                    glm::vec3(basePos.x, basePos.y + 0.35f, basePos.z),     // arriba del tronco
                    glm::vec3(0.75f, 1.2f, 0.75f),                          // copa grande
                    glm::vec3(0.0f, 0.0f, 0.0f),                            // sin rotación local
                    colCopa);                                               // verde oscuro
            };

        // Árbol a la izquierda
        DrawTree(glm::vec3(-3.0f, -0.15f, 0.0f));

        // Árbol a la derecha
        DrawTree(glm::vec3(3.0f, -0.15f, 0.0f));

        // Desactivar shader
        glUseProgram(0);

        // Presentar frame
        mainWindow.swapBuffers();
    }

    return 0;
}
```

Swap buffers

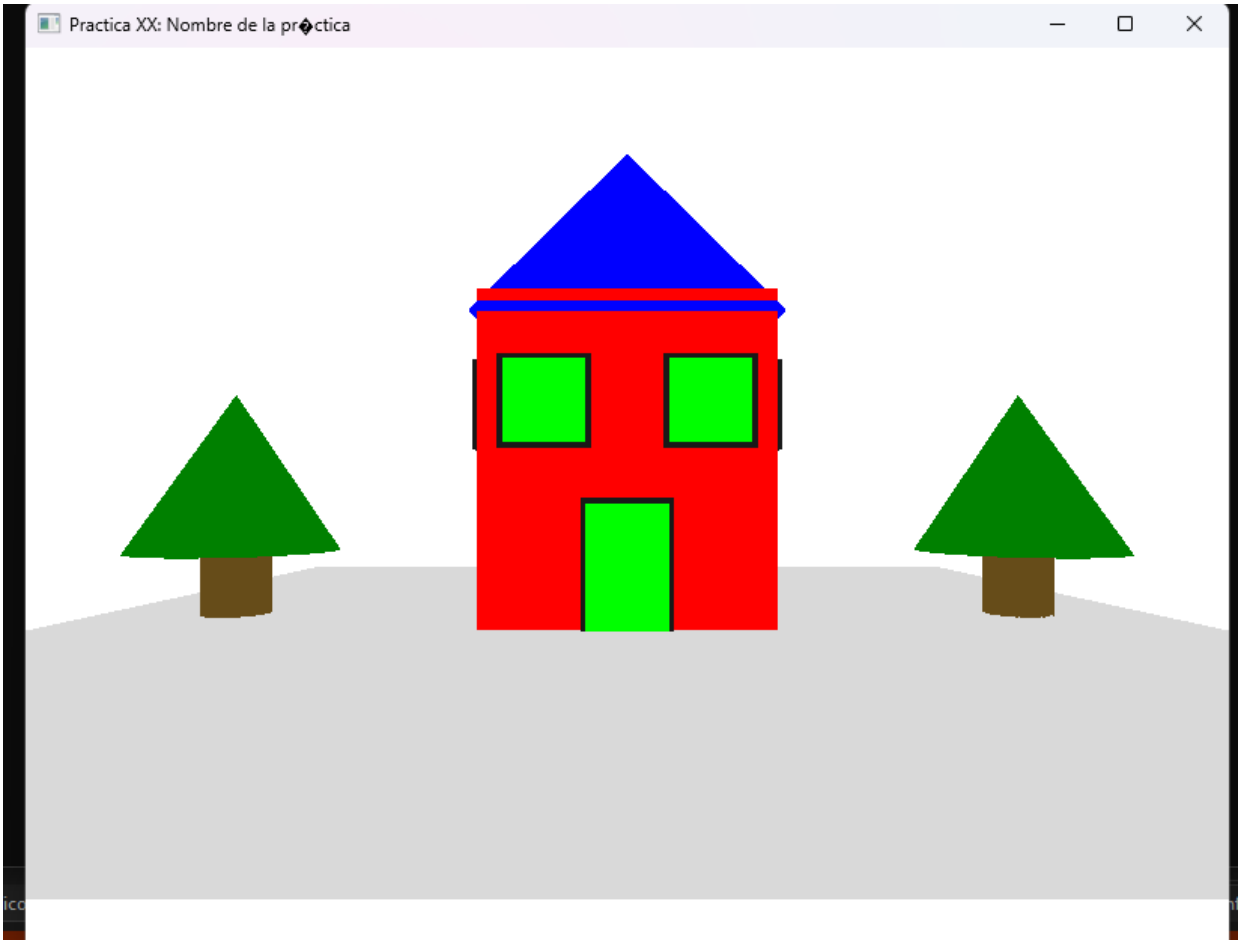
```
mainWindow.swapBuffers();
```

Esto muestra el frame renderizado (doble buffer). Evita parpadeos.

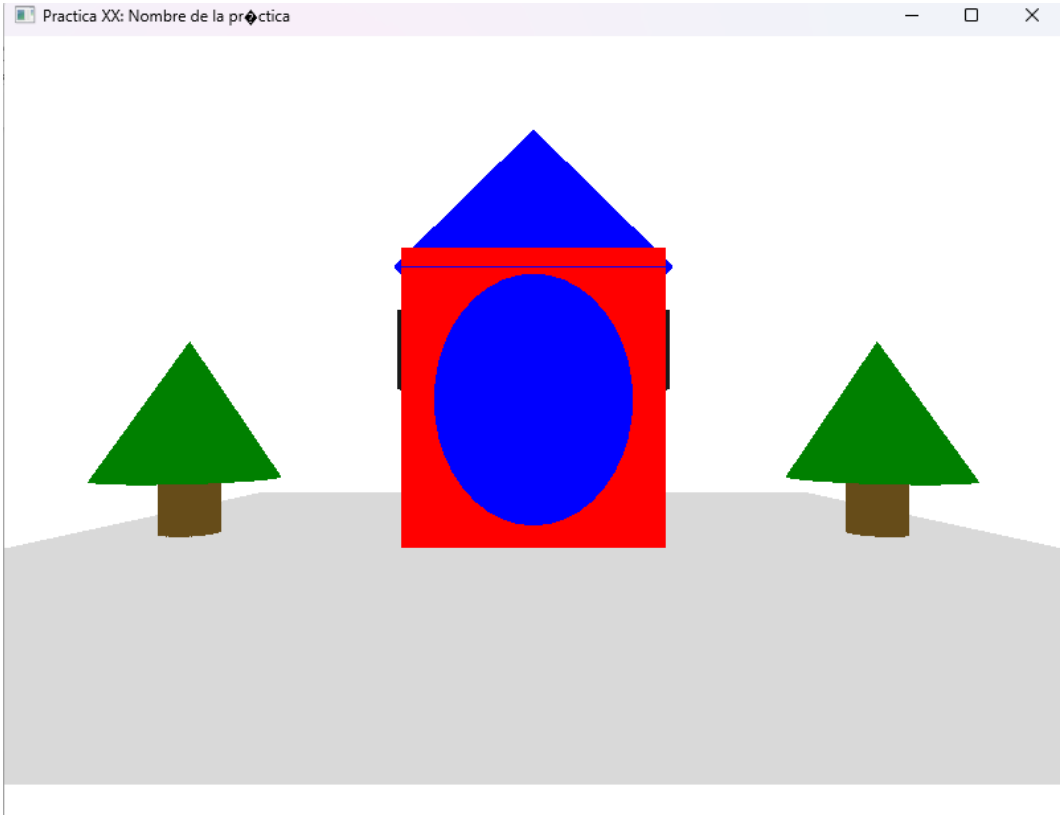
Conclusión

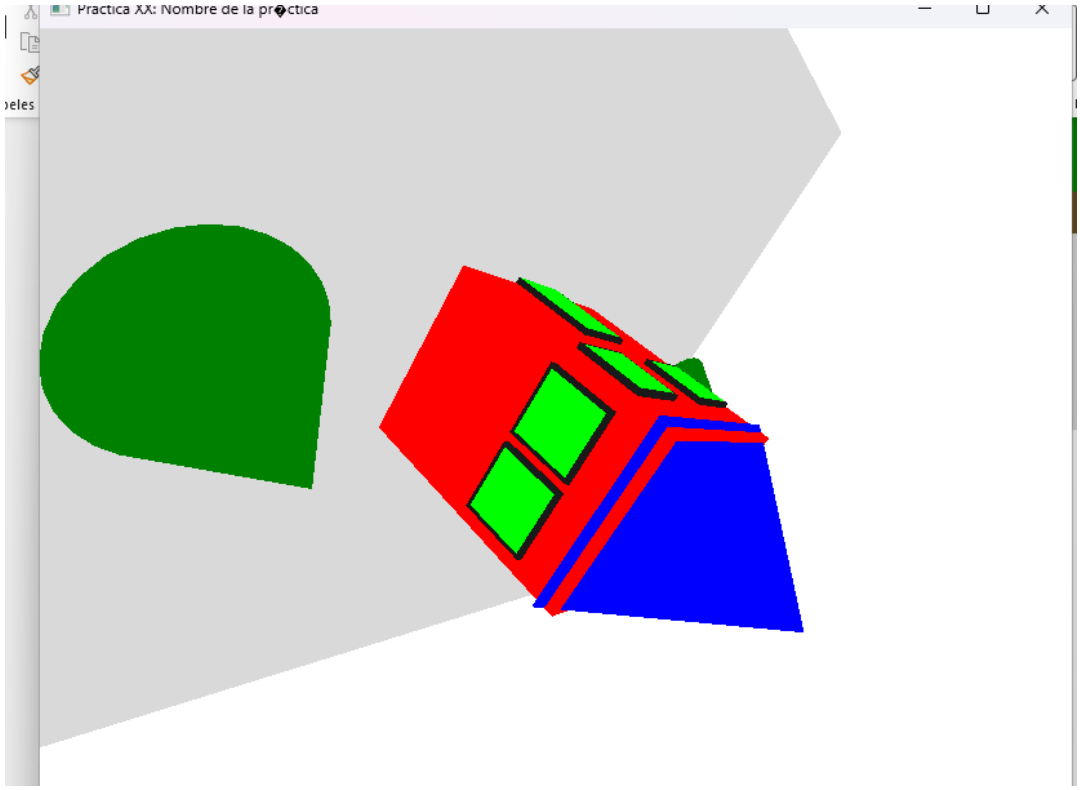
En conjunto, este programa implementa una escena 3D completa usando OpenGL moderno: creo primitivas geométricas básicas (cubo, pirámide, cono, cilindro y esfera), las almaceno como mallas en GPU y luego las instancio varias veces con transformaciones `translate/rotate/scale` para construir un modelo compuesto (la casa con ventanas, puerta, techo, piso y árboles). Además, incluyo una cámara sintética controlada por WASD y mouse y activo Depth Test para que la profundidad se respete correctamente. Con esto logro visualizar el modelo desde distintos ángulos, incluyendo las cuatro vistas que requiere la práctica.

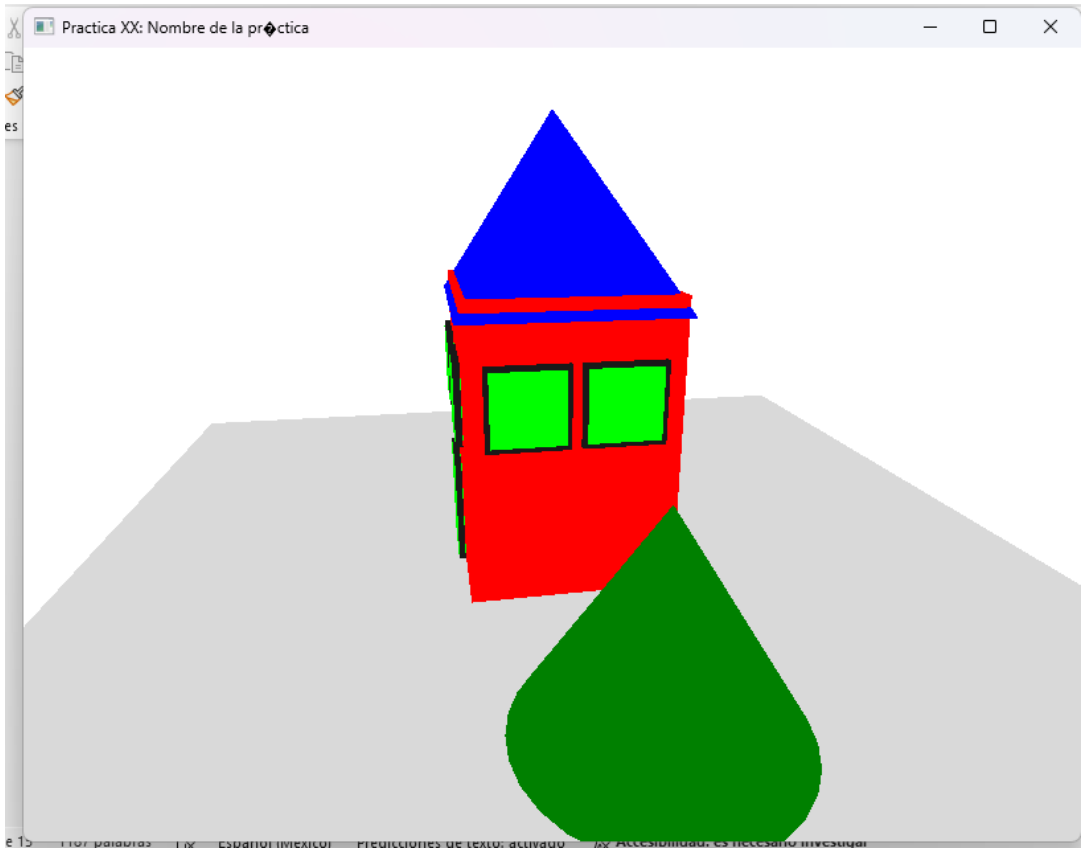
RESULTADO



Practica XX: Nombre de la pr⬚ctica


Practica XX: Nombre de la pr⬚ctica


Practica XX: Nombre de la pr◆ctica


Practica XX: Nombre de la pr◆ctica